

Deep Learning for LDPC Code Decoding in Digital Communications

K. S. Leong

College of Design and Engineering

National University of Singapore

Email: e0725541@u.nus.edu

Abstract—This paper introduces a neural decoder architecture, termed the *ResidualWeightSharingDecoder*, designed for decoding 5G Low-Density Parity-Check (LDPC) codes. The proposed model leverages the structured nature of quasi-cyclic LDPC codes by expanding a base graph into a full parity-check matrix. This structure enables efficient weight sharing between message passing units that correspond to the same entry in the base graph, significantly reducing the number of trainable parameters. In addition, the decoder uses residual connections between iterations, allowing the model to retain and reuse information from previous decoding steps. The experimental results show strong decoding performance and generalization across noise levels.

I. INTRODUCTION

Low-Density Parity-Check (LDPC) codes are a class of powerful error-correcting codes widely deployed in modern digital communication systems, including 5G. Their decoding relies on iterative message-passing algorithms such as Belief Propagation (BP), which are effective but can be limited by high computational cost and lack of adaptability to channel variations.

Recent work has explored neural decoders as a data-driven approach to LDPC decoding, motivated by the structural resemblance between Tanner graphs and neural network architectures. These neural decoders aim to learn decoding strategies directly from noisy codewords, potentially achieving better performance in terms of Bit Error Rate (BER) and Frame Error Rate (FER), particularly under high-noise conditions.

In this work, we present a decoder called the *ResidualWeightSharingDecoder*, which combines insights from traditional message-passing algorithms and deep learning. Incorporate weight sharing across circulant submatrices and residual connections between iterations.

II. RELATED WORK

Classical LDPC decoding relies on belief propagation (BP), which passes messages iteratively between variable and check nodes on the Tanner graph. Extensions such as Min-Sum offer faster approximations but reduce accuracy.

Deep learning approaches, including message-passing neural networks (MPNNs), have shown promise for decoding linear codes. Lucibello et al. [1] proposed deep learning via message passing using BP-like updates. Other works explore

the use of CNNs and residual learning [6], aligning neural models with the sparse structure of LDPC codes.

III. METHODOLOGY

Our decoder is constructed around the Tanner graph representation of the LDPC code. A Tanner graph is a bipartite graph that visually encodes the structure of the LDPC code's parity-check matrix H . The graph contains two types of nodes: *variable nodes* and *check nodes*. Each column of the matrix H corresponds to a variable node, representing a bit in the codeword, while each row corresponds to a check node, representing a parity-check constraint. Decoding is performed using the belief propagation (BP) algorithm, which is an iterative message-passing technique operating over the Tanner graph. In addition to leveraging the structure of the Tanner graph, our proposed decoder enhances the message-passing process with modifications, namely weight sharing and residual connections.

A. Belief Propagation Algorithm

The belief propagation (BP) algorithm is a message-passing technique in which messages represent certain probability functions, such as log-likelihood ratios (LLRs). Any linear code can be visualised using a bipartite Tanner graph, where one set of nodes represents variable nodes (corresponding to the code bits), and the other represents check nodes (corresponding to the parity checks). Each iteration of the algorithm involves two key steps: check node updates and variable node updates. At each iteration, variable nodes update their outgoing messages to check nodes based on the received channel information and incoming check node messages. Check nodes then compute and send back constraint-satisfying messages to variable nodes. This process continues for a fixed number of iterations or until convergence.

a) Traditional Check Nodes Operations:

The standard Sum-Product (SP) update rule is given by:

$$m_{c,v}^{(j,i)}(t+1) = 2 \tanh^{-1} \left(\prod_{k \in V_j, k \neq i} \tanh \left(\frac{m_{v,c}^{(k,j)}(t)}{2} \right) \right) \quad (1)$$

In this expression:

- $m_{c,v}^{(j,i)}(t+1)$ is the message sent from check node j to variable node i at iteration $(t+1)$.

- $m_{v,c}^{(k,j)}(t)$ represents the incoming message from variable node k to check node j at iteration t , excluding the contribution from node i .
- V_j denotes the set of variable nodes connected to check node j , and the condition $k \neq i$ ensures that messages are computed without feedback from the receiving node.

While the Sum-Product Algorithm (SPA) provides optimal decoding performance using hyperbolic functions, it is computationally expensive and difficult to implement efficiently in hardware due to the presence of non-linear operations such as $\tanh$ and $\tanh^{-1}$. To address this, the Min-Sum Algorithm (MSA) is usually used as an approximation, which replaces the hyperbolic tangent product with a sign product and a minimum operation:

$$m_{c,v}^{(j,i)}(t+1) = \prod_{\substack{k \in V_j \\ k \neq i}} \text{sign} \left(m_{v,c}^{(k,j)}(t) \right) \cdot \min_{\substack{k \in V_j \\ k \neq i}} \left| m_{v,c}^{(k,j)}(t) \right| \quad (2)$$

This approximation significantly reduces computational complexity and is more suitable for real-time and hardware-based implementations. Although MSA introduces a small performance degradation compared to SPA, it achieves a favorable trade-off between decoding accuracy and efficiency. In practice, variations such as normalized or offset Min-Sum are also used to further improve the approximation quality.

b) Traditional Variable Nodes Operations:

The variable node update rule, common to both SP and MS algorithms, is expressed as:

$$m_{v,c}^{(i,j)}(t+1) = m_i + \sum_{k \in C_i, k \neq j} m_{c,v}^{(k,i)}(t+1) \quad (3)$$

B. Variable Nodes Update (learnable weights)

$$m_{v,c}^{(i,j)}(t+1) = m_i + \sum_{k \in C_i, k \neq j} m_{c,v}^{(k,i)} w_{base}^k(t+1) \quad (4)$$

This is the variable nodes update rule in the weight sharing and residual message passing algorithm, where:

- m_i is the channel log-likelihood ratio (LLR),
- $m_{c,v}^{(k,i)}$ are the messages from check nodes to variable node i .
- C_i is the set of all check nodes connected to variable node i .
- The summation accumulates messages from check nodes $k \in C_i$ except j ,

C. Check Nodes Update (learnable weights)

$$m_{c,v}^{(j,i)}(t+1) = \prod_{k \in V_j, k \neq i} \text{sign} \left[m_{v,c}^{(k,j)} w_{base}^k(t) \right] \times \min_{\substack{k \in V_j \\ k \neq i}} \left| m_{v,c}^{(k,j)} w_{base}^k(t) \right| \quad (5)$$

This is the check nodes update rule in the weight sharing and residual message passing algorithm, where:

- $m_{v,c}^{(k,j)} w_{base}^k(t)$ is the message (multiplied by shared weight w_{base}^k) sent from the variable node v (indexed by k) to the check node c (indexed by j) in iteration t . It contains the belief (in terms of LLR) about the bit value at variable node v , after considering all other check nodes except c .
- V_j is the set of all variable nodes that are connected to check node j .
- $\prod_{k \in V_j, k \neq i} \text{sign} \left[m_{v,c}^{(k,j)} w_{base}^k(t) \right]$ computes the overall sign of the outgoing messages, which are multiplied by the learnt shared weight from check node j to variable node k (excluding node i from the computation). The product of signs captures the parity-check behavior: if there's an odd number of negative messages, the output is negative.
- $\min_{k \in V_j, k \neq i} \left| m_{v,c}^{(k,j)} w_{base}^k(t) \right|$ selects the smallest absolute value of the incoming messages, which are multiplied by the learnt shared weight, to the check node, again excluding the message from variable node i .

D. Posterior LLR

This operation occurs at the variable node side update when the final iteration has reached. The initial channel LLR and all incoming messages (each multiplied by the learnt shared weights) are summed to obtain the final LLR for every bit of the codeword.

$$l_{D,i}(t+1) = m_i + \sum_{k \in C_i} m_{c,v}^{(k,i)} w_{base}^k(t+1)$$

This represents the final LLR decision after the last iteration, where:

- $l_{D,i}(t+1)$ is the decoded log-likelihood ratio for bit i at iteration $(t+1)$.
- m_i is the channel LLR input
- w_{base}^k is the shared weight from the base graph which is applied to the check messages from check k to variable node i .
- The summation term aggregates **all** incoming messages from check nodes connected to variable node i .

E. Weight Sharing

Weight sharing allows all messages in the expanded circulant submatrix of a base graph, H_{base} , to share the same weight. In other words, a base graph acts as a “template” for a parity-check matrix, which lifts a base graph into a full parity-check matrix through its lifting factor, z . For example, when $z = 2$, we expand each single entry in the base graph to a 2×2 sub-block. The sharing of weights reduces training time as well as the number of trainable parameters and is also more memory efficient.

$$H_{base} = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix}$$

A shift of 0 (entry = 0) yields the 2×2 identity block:

$$I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

A shift of 1 yields:

$$P_1 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

A shift of -1 means “no edge,” so we place

$$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

For a fully expanded matrix with $z = 2$,

$$H_{\text{expanded}} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

F. Residual Connections

Residual connections are used to mitigate the *vanishing gradient problem*, which occurs when gradients become extremely small during backpropagation across many layers. This often prevents earlier layers from learning effectively.

Mathematically, the residual connection can be described as:

$$Y = F(X) + x, \quad (6)$$

where:

- Y is the output at iteration t , which is the sum of $F(X)$ and x .
- $F(X)$ check-to-variable messages from the current iteration, t , multiplied by a learnable weight.
- x are check-to-variable messages from the previous d iterations.

We add the identity, x , which originates from d iterations earlier, to the current computation $F(x)$, where $F(x)$ represents any transformation or series of operations applied to x (depending on the network architecture). This identity connection helps preserve the flow of gradients across layers or iterations. In the context of LDPC decoding, residual connections are applied during the variable node update step at each iteration.

Specifically, for a given residual depth parameter, d , residual connections incorporate information from the past d iterations. For example, if $d = 2$, then at iteration i , the variable node update includes contributions from the check-to-variable messages of iteration $(i - 1)$.

G. Training Procedure

Due to the complexity of the decoder architecture and time constraints, full training of the model could not be completed within the current timeframe. However, substantial effort has been invested in meticulously designing and implementing the architecture, with careful attention to the integration and verification of each part. This includes the generation of training, validation, and test datasets, as well as unit testing

of the message-passing functions that utilises weight-sharing mechanisms and residual connections. All components have been validated and are fully operational, ensuring that the system is prepared and ready for training when computational resources permit. Optimisation uses SGD with momentum = 0.9 and learning rate = 0.01.

IV. CONCLUSION AND FUTURE WORK

This paper presents the ResidualWeightSharingDecoder, a neural architecture for decoding LDPC codes. It efficiently reuses weights through base graph structure and integrates residual connections. Future work includes training a model to test on short codes and long codes and exploring the applicability of shared weights for different codewords with different code rates.

ACKNOWLEDGMENT

The author would like to Professor Zhang Jianwen for his patience, guidance, and kindness throughout the project.

REFERENCES

- [1] C. Lucibello *et al.*, “Deep learning via message passing algorithms based on belief propagation,” *Machine Learning Science and Technology*, vol. 3, no. 3, p. 035005, 2022.
- [2] N. Devroye *et al.*, “Interpreting deep-learned error-correcting codes,” in *Proc. IEEE ISIT*, 2022.
- [3] A. Haroon *et al.*, “Decoding of error correcting codes using neural networks,” in *Proc. Emerging Technologies*, 2013.
- [4] I. Be’ery *et al.*, “Active deep decoding of linear codes,” *arXiv preprint arXiv:1906.02778*, 2019.
- [5] K. S. Leong, “LDPC-NeuralNetwork-Decoder” GitHub. Available: <https://github.com/BananaFalls/LDPC-NeuralNetwork-Decoder>
- [6] “Deep neural network based decoding of short 5G LDPC codes,” *IEEE Conf. Publication*, 2021.